Craft of Programming

Jeff Gates jgates@gmail.com

Diablo III, Blizzard
Spore, Maxis
Dominion, TGG

Please ask questions!

(now, or during the talk, or after, or email)

jgates@gmail.com

Survey

	Familiar	Comfortable
Functional Programming		
Category Theory		
Strongly typed / Weakly typed languages	X	
Databases		X
Garbage collection		
RAII / Smart Pointers		
Unix Philosophy		
Coroutines / Async/Await / Promises		
Shaders / GPU programming		

Programming as Craft

Functional Programming

Databases

The Unix Philosophy

The Prompt model of control flow and Event Driven Programming

Time

Epilogue: Programming as Art

Functional Programming

- My non-official definition:
  - Imperative programming is a sequence of steps we tell the computer
  - Functional programming is a set of relationships we specify to the computer which 'magically' turns into a computation.

- Wikipedia:
  - In computer science, functional programming is a programming paradigm where programs are constructed by applying and composing functions. It is a **declarative** programming paradigm in which function definitions are trees of expressions that map values to other values, **rather than a sequence of imperative statements** which update the running state of the program.

Functional Programming - Math

IP maps to how computers work.
FP maps to math.

```c
// C
int factorial_a(int n) {
     int result = 1;
     for(int i = 2; i < n; ++n) {
          result *= i;
     }
     return result;
}

int factorial_b(int n) {
     if (n == 0) return 1;
     return n * factorial_b(n - 1);
}
```

```haskell
// Haskell
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

// Math

$$n! = \begin{cases} 1 & \text{for } n = 0 \\ n(n\text{-}1)! & \text{for } n > 0 \end{cases}$$

Functional Programing - Pragmatics

- IP is useful for doing things with computers.
- FP is useful for reasoning about programs - ie doing math on programs.
  - CS (after intro) classes tend to be FP
  - IP has too much going on to reason about

Q: So I just said IP is more useful for doing stuff, as a pragmatic programmer, why care about FP?

A: FP provides structure/vocabulary.

- The act of programming is reasoning about a program.
- Practicing FP provides mental shortcuts in that process.
- And as problems get 'very hard' sometimes FP makes them easier.

Functional Programming - Example: fold

```
// Haskell
fold :: (Int -> Int -> Int) -> Int -> [Int] -> Int
```

Let's break it down:

```
  (Int -> Int -> Int) // A function that takes 2 Ints and produces an Int
  Int                 // An initial value
  [Int]               // A list of Integers
  -> Int              // The output Integer
```

```
fold '+' 0 [1, 2, 3]
> 6                // 0  +1 +2 +3
```

$$\sum_{i=1}^{3} i = 6$$

technically,
Σ is `fold '+' 0`
not just `fold`

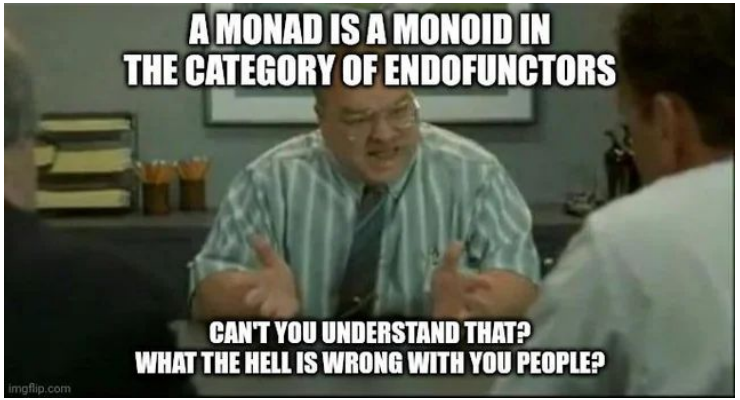
Functional Programming - Folds

```
sum       list = fold (+) 0 list
product   list = fold (*) 1 list
concat    list = fold (++) "" list
```

In IP when you see or need a for-loop that aggregates, you might be able to think 'fold'. Other languages call it 'aggregate' or 'reduce'.

Math-land extends this concept to 'Monoid' in Abstract Algebra and Category Theory.

(the names and abstractions quickly became illegible, as if 'fold' wasn't bad enough)



Functional Programming - Structure vs Flexibility

More structure, clearer reasoning, but less flexibility

- **`map`** - turns a list of As into a list of Bs. N -> N
  - `map (+2) [1, 2, 3]  //  [3, 4, 5]`
- **`filter`** - going to return a list same size or shorter. N -> <= N
  - `filter (<=2) [1, 2, 3]  // [1, 2]`
- **`fold`** - turns a list into a single value. N -> 1
  - `fold (-) 10 [1, 2, 3]  // 10-1-2-3 = 4`

For loop - can do all of the above and more. N -> ???

(aside: similar arguments Strongly vs Weakly typed languages)

Function Programming - pseudocode

```
// pseudo-code
// gain $ filter ((not . (isType Victory)) AND (upTo 5) . cost)

// implementation
void Bargain() {
    CardIDs cards = CardsSupplyTops(SUPPLY_TOPS_FLAG_FOR_GAIN);
    FilterCards([](CardID x) {
        return !CardIs(x, VICTORY) && CostUpto(x, 5);
    }, cards);
    GainOneOf(cards, ChoiceUI(DOMUI_BARGAIN_GAIN_
    OtherPlayers(
        []() { GainCard(CARD_HORSE); }
    );
}
```



Functional Programming - Immutability

Immutability - What if every variable were 'const'
Some things become impossible

```
for(int i = 0; i < 10; ++i) { ... }
```

But some things become easier to understand

```
const int x = y*2;
const int z = x+5;  // always (y*2)+5
```

Expressing IP programs in 'single-static-assignment' can be helpful when debugging

Functional Programming - Pure functions and Side-effects

Pure Functions

```
square :: Int -> Int
square x = x*x
```

$$f(x) = x^2$$

Side Effects

```
void squareAndLog(int x) {
    int result = x*x;
    printf("%d^2=%d", x, result).
    return result;
}
```

```
squareAndLog :: Int -> IO Int
  squareAndLog x = do
    let result = x*x
    printf "%d^2=%d", x, result
    return result
```

Functional Programming - Takeaways

Suggestion: After 'mastering' an imperative language, do 50 Leetcode problems in Haskell.

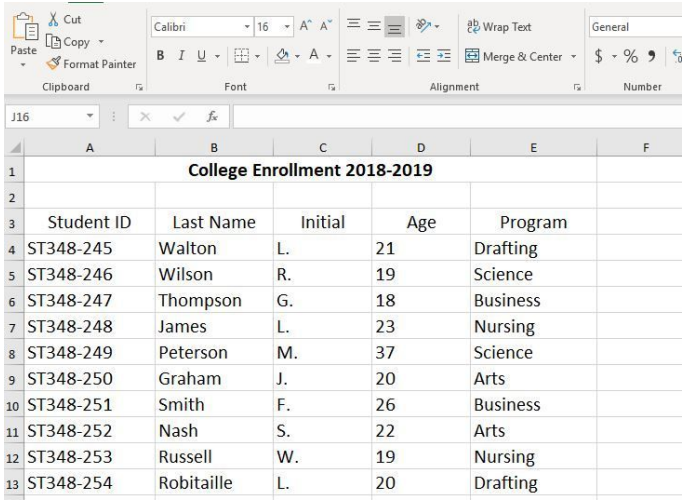
- Build your Vocabulary of common functions
- See 'patterns' (and be able to discern real patterns from snake oil)
- Value pure functions and immutability

Databases

For this talk, database means a relational SQL database, like MySQL.

The interface is roughly:

INSERT INTO *tbl* (*fields, ...*) (*values, ...*)
DELETE FROM *tbl* WHERE *criteria, ...*
UPDATE tbl SET (*field=value, ...*) WHERE *criteria*
SELECT *fields* FROM tbl WHERE *criteria, ...*



	A	B	C	D	E	F
1	College Enrollment 2018-2019					
2						
3	Student ID	Last Name	Initial	Age	Program	
4	ST348-245	Walton	L.	21	Drafting	
5	ST348-246	Wilson	R.	19	Science	
6	ST348-247	Thompson	G.	18	Business	
7	ST348-248	James	L.	23	Nursing	
8	ST348-249	Peterson	M.	37	Science	
9	ST348-250	Graham	J.	20	Arts	
10	ST348-251	Smith	F.	26	Business	
11	ST348-252	Nash	S.	22	Arts	
12	ST348-253	Russell	W.	19	Nursing	
13	ST348-254	Robitaille	L.	20	Drafting	

This is excel, not actually a database, but a good mental picture

Databases - Tables

- Memory management
  - Manual
  - Garbage Collection
  - RAII
  - ... or Tables
- Flat
  - No Pointer-spaghetti - easier to reason about
  - Performance

Databases - Table example

```
struct Player {
     int hp;
     Item items[MAX_ITEMS];
     int numItems;
};

void TradeItem(Player &p1,
     int slot1,
     Player &p2)
{
     Item item = p1.items[slot1];
     RemoveAt(p1.items, p1.numItems, slot1);
     Add(p2.items, p2.numItems, item);
}
```

```
struct Player {
     PlayerID id;
};

struct Item {
     ItemId id;
     PlayerId owner
};

TradeItem(ItemId item, PlayerId p2) {
     Item &item = ItemTable.Lookup(itemId);
     Item.owner = p2;
}
```

Databases - Select / Indexes

The database version
SELECT (fields) FROM (table) WHERE (criteria)

The important concept is the 'criteria'
Implement as simple loop to start - then add an Index

```
FindNearby(Player& p1, float r) {
  for(other : Players) {
     if (dist(player.pos, other.pos) < 10)
      results += other;
  }
  return results;
}
```

```
FindNearby(Player& p1, float r) {
  results = quadtree.query(p1.pos, r);
  for(other : Players) {
    if (dist(player.pos, other.pos) > 10)
      results -= other;
  }
  return results;
}
```

Databases - Select / Indexes Non-games

Web
... = document.querySelectorAll("div...")

Search engine
... = search.lookup(user.location, "restaruants")

Bank
... = transactions.lookup(dateStart, dateEnd)

Social network
... = posts.lookup(friendId).first(20)

Are these all `filter`?

Databases - More

Transactions - Useful to turn individual operations into batches that either succeed or fail as a unit.

Persistence - How/when does the info get saved to disk.

ACID - Atomicity, Consistency, Isolation, and Durability

Databases - Takeaways

Put your data in flat tables
    (or maybe even just think about your data at flat tables)

Isolate queries, and just iterate to start

Build / maintain indexes when necessary
    keep the index structure/data separate

In practice, programs don't often reimplement a 'full database'
(though the SQLLite library gives you that option)
Figure out what pieces your problem needs.

Unix Philosophy

Simple, flexible programs that can be stitched together to do more complicated things.

That 'stitching together' is itself programming. This is a philosophy for a computer designed for programmers. It's not totally in fashion for most computers/OSs today.

Unix Philosophy - Commands

Examples of unix programs that follow this philosophy:

- cat -- inputs a filename, outputs the files contents
- grep -- finds strings
- sed -- replaces strings

And these can be 'stitched together' (real vocab word is 'composed') with pipes:

```
cat logs.txt | grep "error"
```

Wait, is this also **`filter`**?

Unix Philosophy - Criteria

I like to break down programs by answering how it handles these problems:

(answers for the shell and Unix command line tools)

- Input / Output : strings // let's these compose
- State: files (also env variables to an extent)
- Control Flow: none / prompt
- Parallelism: process

Unix Philosophy - Examples

Repl

- Input/Output: strings
- State: env variables
- Control flow: prompt
- parallelism: none

Compiler

- Input/Output: strings/files
- State: none (maybe env variables)
- Control flow: none
- parallelism: process

Web Server

- Input/Output: strings
- State: originally none -> now database
- Control flow: none
- parallelism: original process -> now connection pool

Unix Philosophy - Examples (that start to stray)

Text Editor

- Input/Output: commands
- State: buffers
- Control flow: commands
- parallelism: none?

GPU Programming

- Input/Output: buffers/commands
- State: complicated (but simplifying!)
- Control flow: command list
- parallelism: complicate (was process)

Photoshop

- Input/Output: events, image files
- State: very complicated
- Control flow: commands
- parallelism: none / complicated

Video games

- Input/Output: events, buffers
- State: very complicated
- Control flow: commands
- parallelism: none / complicated

Unix Philosophy - Summary

Criteria: Input/Output, State, Control Flow, Parallelism

Useful to digest what a program is doing, and how it's doing it.

Can use to evaluate our own programs while we develop them

- Can we simplify a criteria
- Can we limit necessary complexity to just one or two criteria

Useful to ideate:

- What is Photoshop but with parallelism: Figma
- What is an LLM but with structured input instead of strings: a new startup?

Prompt

```
while(1) {
    printf("Guess a number: ");
    num = gets();
    if (num == 7) {
        printf("You got it!");
        break;
    } else {
        printf("Try again: ");
    }
}
```

> Guess a number: 42
> Try again: 12
> Try again: 7
> You got it!

Prompt - 'Multiplayer'

```
while(1) {
    nums[] = sendMessageToAll("Guess a number: ");
    some, rest = Filter (==7) nums;
    if (!isEmpty some) {
        map some sendMessage("You got it!");
        map rest sendMessage("Someone else got it!");
    } else {
        printf("Try again: ");
    }
}
```

Prompt

- Ask a question
- Wait for response
- Repeat

Complications:

- What if I have 'broad' question with a lot of possible answers
- How do i 'wait' / what if someone takes a long time to get back to me

Answer: Prompt on Steroids: Event Driven Programming

(or coroutines, or async/await, or promises...)

Event Driven Programming

```
while(1) {
      Event e = WaitForEvent();
      If (e.endProgram) break;
      HandleEvent(e);
}

void HandleEvent(Event e) {
      switch(e.type) {
      case CLICK:
            OnClick(e.clickInfo);
            break;
      case NETWORK_CONNECTION_CLOSED:
            OnNetworkClose(e.networkInfo);
            break;
      //...
      }
}
```

```
void OnClick(ClickInfo) {
      ...
}

void OnNetworkClose(NewtorkInfo) {
      ...
}

...
```

Event Driven Programming - Blocking

```
// Problem: these have to return immediately
void OnClick(ClickInfo) {
     MoveCardToDiscard(idCard);
     // WaitToGetThere(idCard); // Not Allowed
     GivePoints(10);
     If (Score() > 100) EndGame();
}
```

```
// coroutine
void OnClick(ClickInfo) {
     MoveCardToDiscard(idCard);
     while(!AtDiscard(idCard)) {
          yield();
     }
     GivePoints(10);
     If (Score() > 100) EndGame();
}
```

```
// async await
async void OnClick(ClickInfo) {
     await MoveCardToDiscard(idCard);
     GivePoints(10);
     If (Score() > 100) EndGame();
}
```

```
// continuation
void OnClick(ClickInfo) {
     MoveCardToDiscard(idCard).then([]()
          GivePoints(10);
          If (Score() > 100) EndGame();
     };
}
```

Time

Time makes everything about programming harder.

We've seen this with upgrading control flow 'prompt' to 'events'

- Control Flow + Time: Prompt -> Events -> lose 'blocking' -> invent Async -> canceling and timeouts -> ...

Looking at our other criteria:

- Input + Time: servers -> lose composability -> invent streams
- State + Time: transaction logs and histories, mutability, order dependance
- Parallelism + Time: parallelism sort of implies time, so it's already 'hard'

Time - Update loop

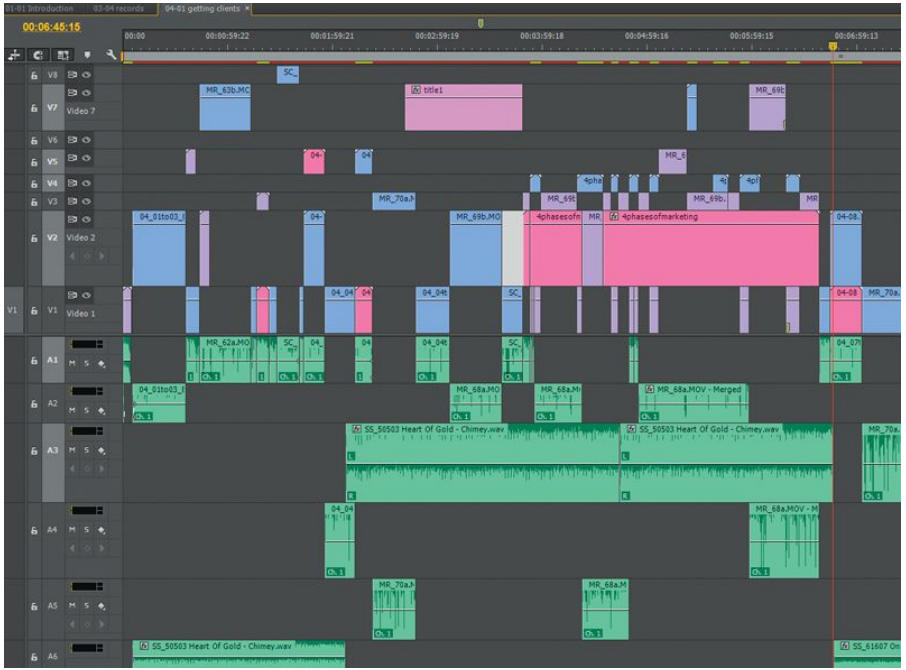
```
float lastUpdate = GetTime();
while(1) {
    Event? e = PollForEvent();
    if (e) {
        if (e.endProgram) break;
        HandleEvent(e);
    }
    float now = GetTime();
    float dt = now - lastUpdate;
    Update(dt);
    lastUpdate = Now;
}
```

Time - Series Data

```
struct {
     float t;
     // Data ...;
};

Song SONG = {
     0, "Mary had a little lamb",
     1, "little lamb",
     2, "little lamb",
     3, "Mary had a little lamb",
     4, "whose fleece was white as snow",
};

SongTracker tracker;
Update(dt) {
  Float oldTime = tracker.time
  Float newTime = tracker.time +dt;
  if (row = Lookup(SONG, oldTime, newTime)) {
     printf(row.text);
  }
  Tracker.time = newTime
}
```



Time - AI

Decision trees

```
DecisionTree Monster = {
     { IfDead(),  FadeOut() },
     { IfEvading(), ContinueEvading() }
     { IfAttacking(), ContinueAttack() }
     { IfMoving(), Continue() }
     { Otherwise, Idle() }
};
```

```
void ContinueAttack() {
     if(!BeingKited(tar))    {
          goal = Evade();
          return;
     }
     if(!InRange(tar))  {
          goal = MoveTo(tar.pos);
          return;
     }
     if(!TargetDead(tar))    {
          if (newTar = FindTarget())
               goal = Attack(newTar);
          else
               goal = NULL;
     }
}
```

Tools

Choose one, become an expert, doesn't matter which

(except it totally does, think iphone/android)

Editor

Vim/Emacs, VSCode

Debugger

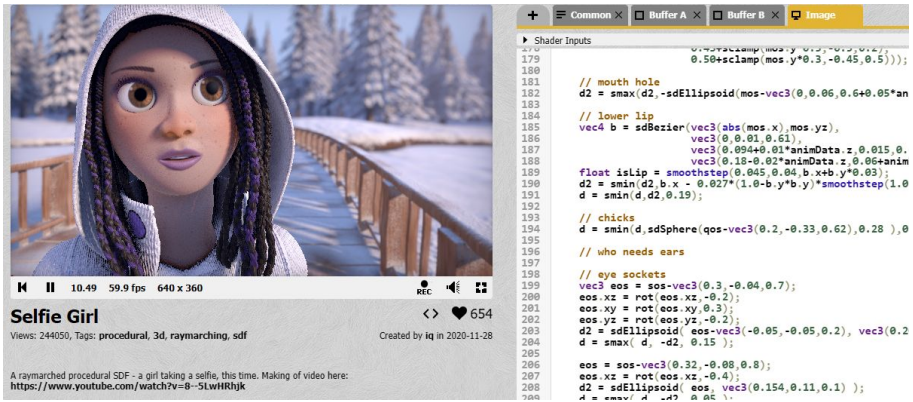
Gdb, windbg, Visual Studio

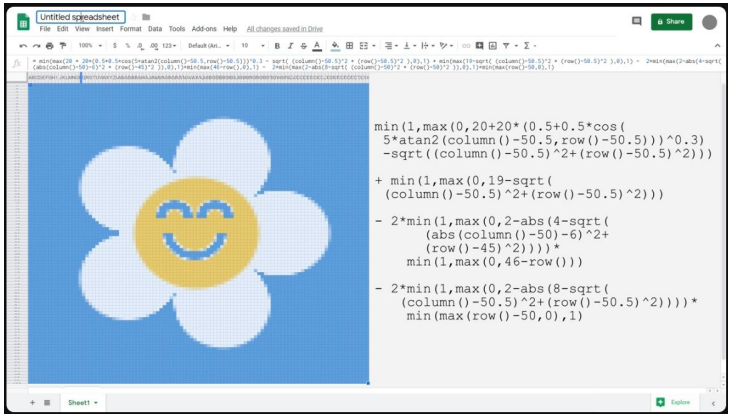
Version Control

Git, Subversion

Programming as Art

Most programming is not art. But some is...





https://www.youtube.com/@InigoQuilez

Programming as Art - Storytime

"Make it skitter"



Programming as Art - Classification

Programming-like Problems

- If it works, it's solved
- Can normally be solved top down - break a big problem into smaller and recurse
- No one cares, best solutions 'disappear'

Art-like Problems

- There is no right answer
- Often a lot of trial and error - 'iteration' if you don't want to sound so negative
- The solution changes the worldview of the person observing it
    - (can be an explicit goal, but often is a side effect)

Programming as Art - Twitter Clone

Most of the tasks are programming-like:

- Have a way to make accounts
- Host on a web server somehow
- UI to let people write tweets
- Store tweets in a database
- Have a way to follow people
- Query the database for people's tweets and display them
- ...

Programming as Art - Twitter Feed

- "Query the database for people's tweets and display them"

The programing-like solution to this would be something like

```
SELECT * from tweets where idAuthor IN MyFollows() ORDER BY date
```

But this isn't the algorithm, Twitter (and other) use. Instead it's 'The Algorithm'.

And I posit 'The Algorithm' is art-like programming

- Has a right answer: no
- Trial and error: yep
- Process prompts questions: almost certainly
- Changes worldview: you betcha

Programming as Art - Responsibility

Art-like problems put a burden on the programmer beyond correctness
They impose responsibility.

That responsibility is often ignored, or 'outsourced' by making a tool
(I love making tools!)

Art-like problems can be afterthoughts of large projects
If 99% of problems on a project are programming-like and 1% art-like,
it can be easy to round that to 0% and easy to overlook the responsibility.

Take it seriously.

Takeaways

- After you get some programming under your belt, try functional programming.
  - (Haskell is a good choice)
- Think about your data as flat tables.
  - Build indexes as needed.
- Try to find solutions that simplify along these axes:
  - Input/Output / State / Control Flow / Parallelism
  - And for control, everything is a Stateless or a Prompt - don't over complicate it!
- Time is hard.
  - Avoid if possible.
  - If not, break things down into sequences.
- Build recognition for programming art-problems. Recognize the responsibility.